Josafat Adrian Hariyanto Copper

Problem 1: The Last Astronaut

```
N, K, size, pos, idx, out, i : integer
circle : array of integer

INPUT N, K
FOR i ← 0 TO N-1 DO
   circle[i] ← i + 1
END FOR

size ← N
pos  ← 0                    // posisi awal hitungan (astronot nomor 1)

WHILE size > 1 DO
   idx ← (pos + K - 1) MOD size     // posisi astronot yang dieliminasi
   out ← circle[idx]
   OUTPUT out                    // catat urutan eliminasi

   // hapus astronot di idx dengan menggeser elemen setelahnya ke kiri
   FOR i ← idx TO size-2 DO
      circle[i] ← circle[i+1]
   END FOR
   size ← size - 1

   // hitungan berikutnya mulai dari astronot tepat setelah yang dieliminasi
   pos ← idx
   IF pos = size THEN
      pos ← 0                  // kembali ke awal lingkaran
   END IF

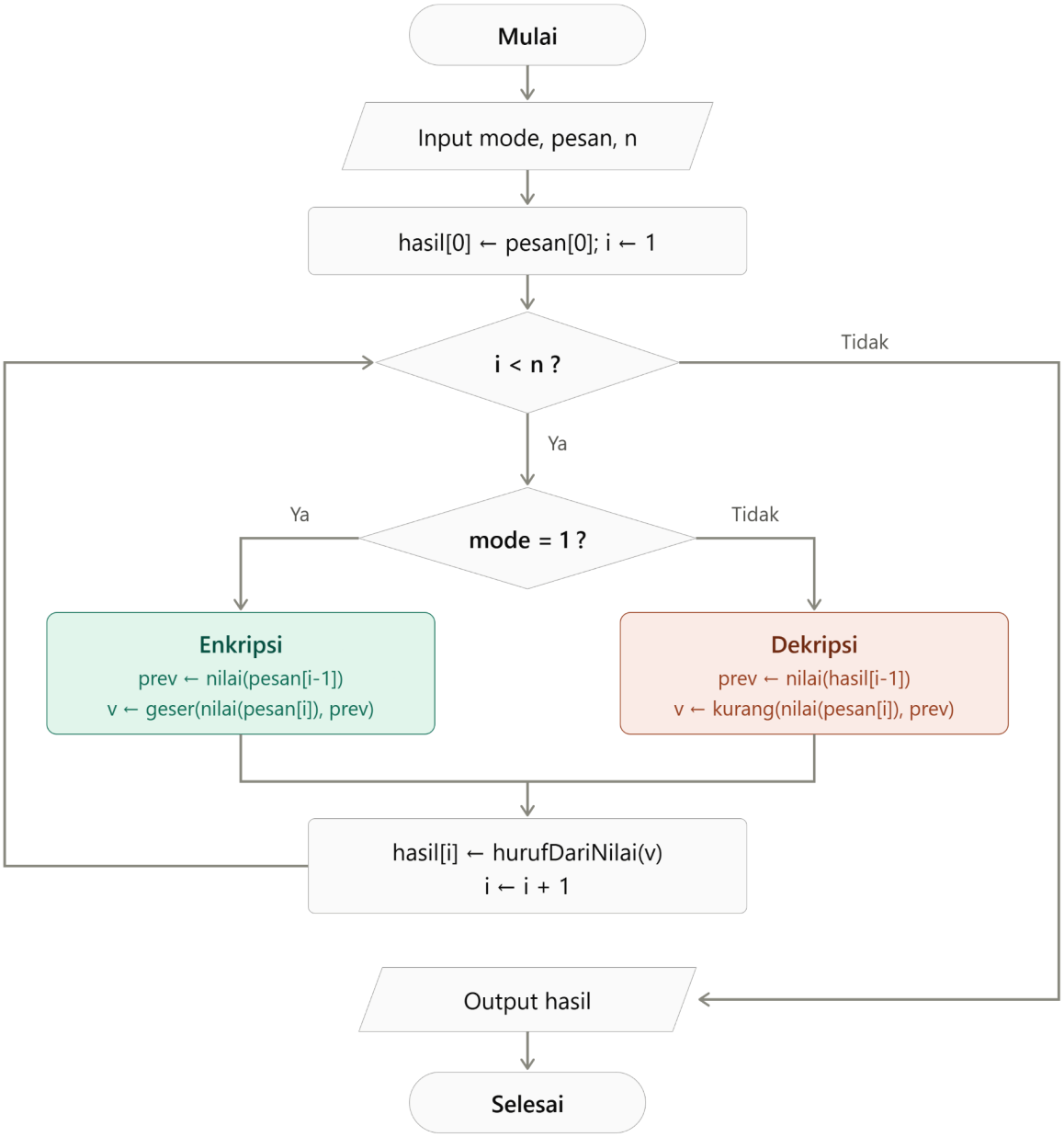
   // update nilai K
   IF out MOD 2 = 0 THEN
      K ← K + 2
   ELSE
      K ← K - 1
   END IF
   IF K < 2 THEN
      K ← 2
   END IF
END WHILE
```

```
    OUTPUT "Astronot terakhir:", circle[0]
SELESAI
```

Lingkaran astronot direpresentasikan dengan array berisi nomor 1 sampai N, dan variabel `pos` menandai dari mana hitungan dimulai. Karena hitungan ke-K dimulai dari `pos` (hitungan ke-1 jatuh pada `pos` itu sendiri), astronot yang dieliminasi berada di indeks `(pos + K - 1) MOD size`. Operator MOD membuat hitungan “memutar” kembali ke awal saat melewati ujung array, sehingga lingkaran tetap terjaga walaupun K lebih besar dari jumlah astronot yang tersisa. Setelah indeks ditemukan, nomor astronot dicetak sebagai bagian dari urutan eliminasi, lalu elemen tersebut dihapus dengan menggeser semua elemen di kanannya satu langkah ke kiri dan mengurangi `size`.

Setelah penghapusan, astronot yang tepat setelah yang dieliminasi otomatis menempati indeks yang sama (`idx`), sehingga `pos ← idx` sudah benar untuk memulai hitungan berikutnya. Satu kasus khusus: jika yang dieliminasi adalah elemen paling akhir, `idx` sama dengan `size` yang baru, jadi `pos` dikembalikan ke 0. Setelah itu, nilai K diperbarui berdasarkan nomor astronot yang baru dieliminasi (bukan indeksnya): genap menambah 2, ganjil mengurangi 1, dan jika hasilnya kurang dari 2 maka K dipaksa menjadi 2. Perulangan berhenti saat `size = 1`, dan satu-satunya elemen yang tersisa di `circle[0]` adalah astronot terakhir.

Problem 2: Alien in the middle



Setiap huruf diubah menjadi angka sesuai posisinya di alfabet (A=1 sampai Z=26) lewat fungsi `nilaiHuruf`. Huruf pertama dibiarkan sama, sedangkan huruf ke-i dienkripsi dengan menjumlahkan nilainya dengan nilai huruf sebelumnya dari pesan asli. Karena hasil penjumlahan bisa melebihi 26, kita pakai wrap-around: `((a + b - 1) MOD 26) + 1`. Pengurangan 1 sebelum MOD dan penambahan 1 sesudahnya menjaga hasil tetap di rentang 1..26, sehingga 26 tidak berubah menjadi 0. Contoh: S(19) + N(14) = 33 → `((33-1) MOD 26)+1 = 7` = G

Untuk dekripsi, prosesnya dibalik: huruf pertama sama, lalu huruf asli ke-i = huruf sandi ke-i dikurangi huruf asli ke-(i-1). Bedanya, huruf “sebelumnya” di sini harus diambil dari hasil dekripsi yang sudah didapat, bukan dari teks sandi, karena yang dipakai saat enkripsi adalah huruf asli. Penambahan 26 di fungsi `kurang` mencegah hasil negatif sebelum MOD. Dengan begitu `AMUNSG` kembali menjadi `ALIENS`.